

D&D Note-Taker — Project Roadmap

Project Vision

A web app for D&D **players** to organize session notes, track characters and NPCs, and build out their knowledge of the game world — with a future expansion path for DMs.

Tech Stack (Beginner-Friendly)

Layer	Tool	Why
Structure	HTML	You already know it
Styling	CSS (+ Tailwind later)	Start vanilla, upgrade when ready
Logic	Vanilla JavaScript	No framework overhead to learn yet
Storage	<code>localStorage</code> → later a backend	Ship fast, migrate later
Hosting	GitHub Pages (free)	Zero config, free HTTPS

Future upgrades (Phase 4+): swap to React, add a Node/Express backend with a database like SQLite or Supabase, and add user accounts.

Architecture Overview

```
dnd-notetaker/
├── index.html      ← Landing / dashboard
├── css/
│   └── style.css   ← All styles
├── js/
│   ├── app.js      ← Router & shared logic
│   ├── storage.js  ← localStorage wrapper (read/write/delete)
│   ├── sessions.js ← Session notes logic
│   ├── characters.js ← Character tracker logic
│   └── world.js     ← World-building logic
├── pages/
│   ├── sessions.html
│   └── characters.html
```

```
| └─ world.html
└─ assets/
   └─ icons/
```

This is a **multi-page app** (simplest pattern). Each HTML page is its own screen. `storage.js` is the single source of truth for saving/loading data.

Phase 1 — Foundation (Week 1–2)

Goal: Get a working skeleton with navigation and basic storage.

Tasks

1. Project setup

- Create the folder structure above.
- Create `index.html` as a dashboard with links to the three sections (Sessions, Characters, World).
- Add a shared `<nav>` bar at the top of every page.

2. Styling basics

- Pick a dark-fantasy color palette (parchment backgrounds, dark navbars).
- Set up CSS variables: `--color-bg`, `--color-text`, `--color-accent`, etc.
- Style the nav bar and page layout so it looks consistent.

3. Storage module (`storage.js`)

- Write helper functions that wrap `localStorage`:

```
js

// Example API to build
saveItem(key, data) // JSON.stringify + store
loadItem(key)       // retrieve + JSON.parse
deleteItem(key)     // remove
listItems(prefix)   // get all keys starting with "session_", etc.
```

- This module is the backbone — every feature will use it.

4. Dashboard (`index.html`)

- Show a simple summary: number of sessions logged, number of characters, etc.
- Add quick-links / cards to each section.

What You'll Learn

- Organizing a multi-file JS project
 - Using `localStorage` for persistence
 - CSS custom properties (variables)
-

Phase 2 — Session Notes (Week 3–4)

Goal: Players can create, edit, and browse session journals.

Data Model

```
js
{
  id: "session_1715000000000",
  title: "The Mines of Kharduun",
  date: "2025-03-01",      // real-world session date
  inGameDate: "14th of Mirtul", // optional
  summary: "We explored the abandoned dwarven mine...",
  tags: ["combat", "loot", "Kharduun"],
  createdAt: 1715000000000
}
```

Tasks

1. Session list page (`sessions.html`)

- Load all sessions from storage and display as a list of cards.
- Each card shows: title, date, first ~100 characters of summary.
- Add a "New Session" button at the top.

2. Create / Edit form

- A simple form with fields for title, date, in-game date, summary (textarea), and tags (comma-separated input).
- On save, generate a unique ID (`"session_" + Date.now()`), call `saveItem()`.

3. Session detail view

- Click a card → show the full session note.
- Add "Edit" and "Delete" buttons.

4. Search and filter

- A search bar that filters sessions by title or tag.
- Optional: sort by date (newest first / oldest first).

What You'll Learn

- Rendering dynamic lists from data (DOM manipulation)
 - Handling form inputs and events
 - Basic CRUD (Create, Read, Update, Delete) patterns
-

Phase 3 — Character Tracker (Week 5–7)

Goal: Track PCs, NPCs, and their relationships.

Data Model

```
js
{
  id: "char_1715000000000",
  name: "Theron Ashvale",
  type: "NPC",           // "PC" or "NPC"
  race: "Half-Elf",
  class: "Ranger",       // nullable for NPCs
  location: "Silvermoon",
  status: "alive",        // alive | dead | missing | unknown
  description: "A grizzled tracker with a scar across...",
  notes: "Helped us in Session 3. Owes a debt to...",
  tags: ["ally", "quest-giver"],
  linkedSessions: ["session_1715000000000"],
  createdAt: 1715000000000
}
```

Tasks

1. **Character list page** (characters.html)
 - Display all characters as cards or a table.
 - Add filter tabs: "All / PCs / NPCs".
 - Add a status badge (alive = green, dead = red, etc.).
2. **Create / Edit form**
 - Fields for each data model property.

- Dropdown for `type` and `status`.
- A multi-select or tag input for `tags`.

3. Character detail view

- Full character info.
- Show "Linked Sessions" as clickable links to session notes.

4. Linking characters ↔ sessions

- When editing a session, add a "Characters Involved" picker.
- When viewing a character, show which sessions mention them.
- This is your first **cross-reference** — a key UX feature.

What You'll Learn

- Working with related data across different collections
 - Building filter/tab UIs
 - Dropdown and multi-select form patterns
-

Phase 4 — World-Building (Week 8–10)

Goal: Organize locations, factions, lore, and items.

Data Model (Locations example)

```
js
{
  id: "loc_1715000000000",
  name: "The Sunken Citadel",
  type: "dungeon",          // city | dungeon | region | tavern | wilderness | other
  parentLocation: "loc_...", // nullable — allows nesting (city inside a region)
  description: "A crumbling fortress half-submerged in...",
  notes: "We haven't fully explored Level 3 yet.",
  tags: ["dangerous", "undead"],
  linkedCharacters: ["char_..."],
  linkedSessions: ["session_..."],
  createdAt: 1715000000000
}
```

Tasks

1. World page (`world.html`)

- A tabbed view: Locations | Factions | Lore | Items.
- Each tab shows a filterable list of entries.

2. Location hierarchy

- Allow nesting: a Tavern can be inside a City which is inside a Region.
- Display as an indented list or a simple tree view.

3. Cross-linking everything

- Locations link to characters and sessions.
- Clicking any linked item navigates to its detail page.
- This is the "wiki-like" feel that makes the app powerful.

4. Quick-add from session notes

- While writing a session note, let the user highlight a name and quickly create a stub character or location from it (stretch goal).

What You'll Learn

- Recursive/nested data structures (parent-child locations)
 - Tab-based navigation within a page
 - Building a connected knowledge base
-

Phase 5 — Polish & Quality of Life (Week 11–12)

Goal: Make it feel like a real app, not a school project.

Tasks

1. Theming

- Add a parchment / fantasy aesthetic: textured backgrounds, fantasy fonts (Google Fonts: "MedievalSharp", "Cinzel", "IM Fell English").
- Dark mode toggle.

2. Data export / import

- "Export All Data" button → downloads a `.json` file.
- "Import Data" button → uploads and merges.

- This is your backup system until you have a real backend.

3. Markdown support in notes

- Let users write session summaries in Markdown.
- Use a lightweight library like `marked.js` to render it.

4. Responsive design

- Make sure it works well on phones and tablets (players often use these at the table).

5. Keyboard shortcuts

- `Ctrl+N` for new session, `Ctrl+S` to save, `/` to focus search.

Phase 6 — Future Expansion (DM Features)

Once the player-facing app is solid, here's how to grow it.

Feature	Description
DM mode toggle	A switch that reveals DM-only fields (secret NPC motivations, hidden lore, encounter prep).
Encounter planner	Create encounters with monsters, CR calculation, and initiative order.
Timeline view	A visual timeline of in-game events across sessions.
Maps	Upload an image and place pins for locations. Libraries: Leaflet.js with a custom image layer.
User accounts + cloud sync	Move from localStorage to a backend (Supabase is free and beginner-friendly).
Shared campaigns	DM shares a campaign code → players join and see shared world info.

Suggested Coding Order (Checklist)

Use this as a to-do list. Check items off as you go.

Phase 1 — Foundation

- ☐ Create folder structure
- ☐ Build index.html with nav
- ☐ Write storage.js helpers
- ☐ Style the layout and nav (CSS variables)
- ☐ Build the dashboard with summary cards

Phase 2 — Session Notes

- ☐ Session list page (load + render from storage)
- ☐ New session form (create + save)
- ☐ Session detail view (read)
- ☐ Edit session (update)
- ☐ Delete session (delete + confirm dialog)
- ☐ Search / filter bar

Phase 3 — Character Tracker

- ☐ Character list page with filter tabs
- ☐ New character form
- ☐ Character detail view
- ☐ Link characters ↔ sessions (cross-referencing)
- ☐ Status badges

Phase 4 — World-Building

- ☐ World page with tabs (Locations, Factions, Lore, Items)
- ☐ Location CRUD
- ☐ Location hierarchy (nesting)
- ☐ Cross-links across all entities

Phase 5 — Polish

- ☐ Fantasy theming + dark mode
- ☐ Export / import JSON
- ☐ Markdown rendering in notes
- ☐ Responsive mobile layout
- ☐ Keyboard shortcuts

Tips for a Beginner

- **Build ugly first, then make it pretty.** Get the functionality working before spending time on CSS.
- **One feature at a time.** Don't jump between phases. Finish session notes before starting characters.
- **Use the browser console.** `console.log()` is your best debugging friend.

- **Commit to Git often.** Even if you're new to Git, learning `git add`, `git commit`, and `git push` now will save you pain later.
 - **Test your storage early.** Open DevTools → Application → Local Storage to see your data. If it looks wrong, fix `storage.js` before building more features on top of it.
 - **Don't be afraid to refactor.** Your Phase 1 code will look different by Phase 3. That's normal and good.
-

Good luck, adventurer. May your notes be organized and your sessions legendary. 🧙‍♂️